

# Java DBMS interview questions

intermediate level questions on java interview and dbms

Here's a structured list of **intermediate-level Java and DBMS interview questions** you can practice. I've separated them into categories so it's easier to focus. I've also included some brief hints/answers for each so you can self-evaluate.

---

## Java Intermediate Interview Questions

### Core Java

1. What is the difference between `==` and `equals()` in Java?

*Hint:* `==` compares references; `equals()` compares content.

2. What are **Wrapper Classes**? Why are they needed?

*Hint:* Converts primitive types into objects. Example: `int` → `Integer`.

3. Explain **Autoboxing** and **Unboxing**.

*Hint:* Automatic conversion between primitives and wrapper classes.

4. Difference between **ArrayList** and **LinkedList**.

*Hint:* ArrayList is backed by array (fast random access), LinkedList by nodes (fast insert/delete).

5. What is **HashMap**? How does it work internally?

*Hint:* Stores key-value pairs using a hash table. Handle collisions via chaining (or tree in Java 8+).

6. Difference between **HashMap**, **LinkedHashMap**, and **TreeMap**.

*Hint:* Order maintenance, sorting, and insertion order.

7. What is the difference between **abstract class** and **interface**?

*Hint:* Abstract can have methods with implementations, interface in Java 8+ can have default methods, supports multiple inheritance.

8. Explain **final**, **finally**, and **finalize**.

*Hint:* Keyword, block, method for GC cleanup.

9. Difference between **throw** and **throws**.

*Hint:* `throw` used to throw an exception, `throws` declared in method signature.

10. What is **synchronized** in Java? How does it affect threads?

*Hint:* Locks a block/method for one thread at a time.

---

## Java OOPs Concepts

11. What is **polymorphism**? Give examples of compile-time and runtime polymorphism.
  12. Explain **encapsulation** and how Java supports it.
  13. Difference between **composition** and **inheritance**.
  14. Explain **method overloading** vs **method overriding**.
  15. What is **constructor chaining** in Java?
- 

## Java Collections & Streams

16. Difference between **Iterator**, **ListIterator**, and **Enumeration**.
  17. How does **ConcurrentHashMap** differ from **HashMap**?
  18. What are **functional interfaces**? Example?
  19. Difference between **map()**, **flatMap()**, and **filter()** in Java streams.
  20. How does **Optional** help avoid **NullPointerException**?
- 

## Multithreading & Concurrency

21. Difference between **Runnable** and **Thread**.
  22. Explain **Thread Pool** and why it's better than creating threads manually.
  23. What is **volatile** keyword?
  24. Difference between **synchronized block** and **synchronized method**.
  25. What is **Deadlock**, and how can it be avoided?
- 

## DBMS Intermediate Interview Questions

### SQL Queries

1. Difference between **WHERE** and **HAVING** clauses.
2. What are **joins**? Difference between **INNER**, **LEFT**, **RIGHT**, and **FULL OUTER JOIN**.
3. Difference between **UNION** and **UNION ALL**.
4. Difference between **TRUNCATE**, **DELETE**, and **DROP**.
5. How does **GROUP BY** work?
6. Explain **subquery** vs **correlated subquery**.
7. Difference between **Primary Key**, **Foreign Key**, and **Unique Key**.
8. What are **indexes**? How do they improve performance?
9. Explain **ACID properties** of a transaction.

## 10. Difference between **CHAR** and **VARCHAR**.

---

## Advanced SQL & DB Concepts

11. What is **normalization**? Difference between 1NF, 2NF, 3NF.
  12. What is a **view**? Difference between **materialized view** and regular view.
  13. Explain **stored procedures** vs **functions** in DB.
  14. Difference between **inner** and **outer transactions**.
  15. What is **locking**? Difference between **pessimistic** and **optimistic locking**.
  16. Explain **foreign key constraint cascade options** (ON DELETE CASCADE, ON UPDATE CASCADE).
  17. Difference between **clustered** and **non-clustered index**.
  18. What is **DBMS vs RDBMS**?
  19. Difference between **OLTP** and **OLAP**.
  20. Explain **database normalization vs denormalization** with examples.
- 

If you want, I can make a **ready-to-practice set of 50 mixed Java + DBMS intermediate questions with answers**, so you can simulate an actual interview session.

Do you want me to do that?

What is constructor chaining in Java?

**Constructor chaining** in Java is the process where a **constructor calls another constructor** in the same class or in the parent class to reuse code and avoid duplication. This ensures that initialization is done in a structured way.

There are **two types of constructor chaining**:

---

### 1. Within the same class

- A constructor can call another constructor in the **same class** using the keyword `this()`.
- Rule: The call to `this()` **must be the first statement** in the constructor.

**Example:**

java

```
class Person {
    String name;
    int age;

    // Constructor 1
    Person() {
        this("Unknown", 0); // calls Constructor 2
        System.out.println("No-arg constructor called");
    }

    // Constructor 2
    Person(String name, int age) {
        this.name = name;
        this.age = age;
        System.out.println("Parameterized constructor called");
    }
}

public class Main {
    public static void main(String[] args) {
        Person p = new Person();
    }
}
```

## Output:

pgsql

Parameterized constructor called  
No-arg constructor called

## 2. Between parent and child class

- A child class constructor can call the parent class constructor using `super ( )` .
- Rule: The call to `super ( )` **must also be the first statement** in the constructor.

## Example:

java

```
class Animal {
    Animal() {
        System.out.println("Animal constructor");
    }
}

class Dog extends Animal {
    Dog() {
        super(); // calls Animal constructor
        System.out.println("Dog constructor");
    }
}

public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
    }
}
```

### Output:

```
kotlin
```

```
Animal constructor
```

```
Dog constructor
```

---

### ✓ Key Points:

1. Constructor chaining avoids duplicate code.
  2. `this()` calls constructors in the same class.
  3. `super()` calls constructors in the parent class.
  4. Both `this()` and `super()` must be the **first statement** in a constructor.
- 

If you want, I can draw a **small diagram showing how constructor chaining flows**, which makes it much easier to remember.

Do you want me to do that?

## Difference between Iterator, ListIterator, and Enumeration.

Here's a clear comparison of **Iterator**, **ListIterator**, and **Enumeration** in Java:

| Feature                       | Enumeration                                                      | Iterator                                                                            | ListIterator                                                                                                                                                          |
|-------------------------------|------------------------------------------------------------------|-------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Introduced in                 | JDK 1.0                                                          | JDK 1.2                                                                             | JDK 1.2                                                                                                                                                               |
| Applicable to                 | Legacy classes like <code>Vector</code> , <code>Hashtable</code> | All <b>Collection</b> classes (like <code>ArrayList</code> , <code>HashSet</code> ) | Only <b>List</b> classes (like <code>ArrayList</code> , <code>LinkedList</code> )                                                                                     |
| Methods                       | <code>hasMoreElements()</code> , <code>nextElement()</code>      | <code>hasNext()</code> , <code>next()</code> , <code>remove()</code>                | <code>hasNext()</code> , <code>next()</code> , <code>hasPrevious()</code> , <code>previous()</code> , <code>add()</code> , <code>remove()</code> , <code>set()</code> |
| Traversal Direction           | Forward only                                                     | Forward only                                                                        | Forward and backward                                                                                                                                                  |
| Modification during iteration | No <code>remove()</code> support                                 | Can remove elements using <code>remove()</code>                                     | Can modify list: add, remove, update ( <code>set()</code> )                                                                                                           |
| Fail-fast                     | No                                                               | Yes (throws <code>ConcurrentModificationException</code> )                          | Yes                                                                                                                                                                   |

### Key Differences in Words

1. **Enumeration** is **legacy** and only allows **forward traversal**, cannot remove elements.
2. **Iterator** is the modern replacement; allows **forward traversal** and safe **removal**.
3. **ListIterator** is more powerful; works **only with lists**, allows **both forward and backward traversal**, and supports **adding, removing, and updating elements** during iteration.

### Example Usage:

```
java
```

```

import java.util.*;

public class Test {
    public static void main(String[] args) {
        List<String> list = new ArrayList<>(Arrays.asList("A", "B", "C"));

        // Iterator
        Iterator<String> it = list.iterator();
        while(it.hasNext()) {
            System.out.println(it.next());
        }

        // ListIterator
        ListIterator<String> lit = list.listIterator();
        while(lit.hasNext()) {
            System.out.println("Forward: " + lit.next());
        }
        while(lit.hasPrevious()) {
            System.out.println("Backward: " + lit.previous());
        }

        // Enumeration (legacy)
        Vector<String> vector = new Vector<>(list);
        Enumeration<String> en = vector.elements();
        while(en.hasMoreElements()) {
            System.out.println(en.nextElement());
        }
    }
}

```

If you want, I can also make a **tiny flow diagram showing traversal and modification abilities** of all three—it's easier to visualize for interviews.

Do you want me to make it?

How does ConcurrentHashMap differ from HashMap?

Here's a clear comparison between **HashMap** and **ConcurrentHashMap** in Java:

| Feature          | HashMap                                                                                                                    | ConcurrentHashMap                                                                                                                                            |
|------------------|----------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Thread Safety    | Not thread-safe. Concurrent modifications can lead to unpredictable behavior.                                              | Thread-safe. Multiple threads can read and write concurrently without corrupting the map.                                                                    |
| Synchronization  | None. Needs external synchronization if used in multi-threaded environment ( <code>Collections.synchronizedMap()</code> ). | Uses internal segmentation or locking mechanism (free reads in Java 8+). Synchronization is handled internally.                                              |
| Null Keys/Values | Allows <b>one null key</b> and multiple null values.                                                                       | Does <b>not allow null keys or null values</b> .                                                                                                             |
| Performance      | Fast in single-threaded environment.                                                                                       | Slightly slower than HashMap in single-threaded environment due to synchronization overhead. Faster than synchronized HashMap in multi-threaded environment. |
| Iteration        | Fail-fast iterator. Throws <code>ConcurrentModificationException</code> if modified during iteration.                      | Weakly consistent iterator. Does <b>not throw</b> <code>ConcurrentModificationException</code> ; reflects not necessarily all modifications.                 |
| Use Case         | Single-threaded applications, or when thread safety is externally handled.                                                 | Multi-threaded applications where multiple threads read/write the map concurrently.                                                                          |

## Key Points:

1. **HashMap**: Not safe for concurrent modification. Use external locks if needed.
2. **ConcurrentHashMap**: Designed for **high concurrency**; uses internal partitioning and lock-free reads in Java 8+.
3. **Null Handling**: HashMap allows `null` , ConcurrentHashMap does not.

## Example:

```
java

import java.util.*;
import java.util.concurrent.*;

public class Test {
```

```

public static void main(String[] args) {
    HashMap<Integer, String> hashMap = new HashMap<>();
    hashMap.put(1, "A");
    hashMap.put(null, "B"); // allowed
    System.out.println(hashMap);

    ConcurrentHashMap<Integer, String> conMap = new ConcurrentHashMap<>();
    conMap.put(1, "A");
    // conMap.put(null, "B"); // throws NullPointerException
    System.out.println(conMap);
}
}

```

If you want, I can make a **visual diagram showing thread-safety differences** between `HashMap`, `SynchronizedMap`, and `ConcurrentHashMap`. It's very handy for interviews.

Do you want me to do that?

Difference between `map()`, `flatMap()`, and `filter()` in Java streams.

Here's a clear explanation of `map()`, `flatMap()`, and `filter()` in Java Streams, with examples:

| Method                 | Purpose                                                                                         | Input → Output                                         | Example                                                                                           |
|------------------------|-------------------------------------------------------------------------------------------------|--------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| <code>map()</code>     | Transforms each element of a stream into another form                                           | <code>T → R</code> (one-to-one mapping)                | Convert <code>List&lt;String&gt;</code> of names to their lengths                                 |
| <code>flatMap()</code> | Transforms each element into a <b>stream</b> and then flattens all streams into a single stream | <code>T → Stream&lt;R&gt;</code> (one-to-many mapping) | Convert <code>List&lt;List&lt;Integer&gt;&gt;</code> into a flat <code>List&lt;Integer&gt;</code> |
| <code>filter()</code>  | Filters elements based on a <b>predicate</b>                                                    | <code>T → boolean</code>                               | Keep only even numbers from a list                                                                |

---

## Examples

### 1. `map()`

java

```
List<String> names = Arrays.asList("Alice", "Bob", "Charlie");
List<Integer> lengths = names.stream()
    .map(String::length) // Transform each name to its length
    .toList();
System.out.println(lengths); // Output: [5, 3, 7]
```

### 2. `flatMap()`

java

```
List<List<Integer>> listOfLists = Arrays.asList(
    Arrays.asList(1, 2),
    Arrays.asList(3, 4),
    Arrays.asList(5)
);

List<Integer> flatList = listOfLists.stream()
    .flatMap(List::stream) // Flatten nested lists
    .toList();
System.out.println(flatList); // Output: [1, 2, 3, 4, 5]
```

### 3. `filter()`

java

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6);
List<Integer> evenNumbers = numbers.stream()
    .filter(n -> n % 2 == 0) // Keep only even numbers
    .toList();
System.out.println(evenNumbers); // Output: [2, 4, 6]
```

---

## Key Points

1. `map()` → **one-to-one** transformation.

2. `flatMap()` → **one-to-many** transformation; flattens nested streams.
  3. `filter()` → **selection** based on a boolean condition.
- 

If you want, I can also make a **tiny diagram showing map vs flatMap vs filter flow**, which makes it very easy to remember for interviews.

Do you want me to make it?

How does Optional help avoid NullPointerException?

`Optional` in Java is a container object that **may or may not contain a value**. It helps **avoid** `NullPointerException` (**NPE**) by making the possibility of a missing value explicit, rather than dealing with `null` directly.

---

## Key Benefits

1. Forces the programmer to **check for presence** before accessing the value.
  2. Provides **functional-style methods** like `map()`, `filter()`, `ifPresent()`, and `orElse()` to safely handle null cases.
  3. Makes the code **more readable and less error-prone** than using plain `null`.
- 

## Example 1: Traditional Null Check

```
java

String name = getName(); // might return null

if(name != null) {
    System.out.println(name.toUpperCase());
} else {
    System.out.println("Name not found");
}
```

---

## Example 2: Using Optional

java

```
Optional<String> nameOpt = Optional.ofNullable(getName());

nameOpt
    .map(String::toUpperCase) // transforms the value if present
    .ifPresentOrElse(         // acts only if value is present
        System.out::println,
        () -> System.out.println("Name not found")
    );
```

### Explanation:

- `Optional.ofNullable(value)` → Wraps a value that might be null.
- `map()` → Safely transforms the value if it exists.
- `ifPresentOrElse()` → Executes a block if value exists, otherwise executes alternative.

---

### Other Useful Optional Methods

- `orElse(defaultValue)` → Returns value or default if absent.
- `orElseGet(() -> supplier)` → Returns value or computes default lazily.
- `orElseThrow(() -> exception)` → Throws exception if value absent.

---

### Summary

- **Before Optional:** Direct `null` checks → error-prone, scattered code.
- **With Optional:** Explicit, safe, readable handling → reduces NPEs.

---

If you want, I can also make a **tiny diagram showing the flow of Optional methods**—it's really handy for interviews to visualize how it avoids NPE.

Do you want me to make it?

Difference between Runnable and Thread.

Here's a detailed comparison between `Runnable` and `Thread` in Java:

| Feature                          | Runnable                                                                                                  | Thread                                                                               |
|----------------------------------|-----------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| Definition                       | <code>Runnable</code> is a <b>functional interface</b> that represents a task to be executed in a thread. | <code>Thread</code> is a <b>class</b> that represents a thread of execution in Java. |
| How to use                       | Implement <code>Runnable</code> and pass it to a <code>Thread</code> object.                              | Extend the <code>Thread</code> class and override its <code>run()</code> method.     |
| Multiple inheritance             | Can be used with <b>other classes</b> (because Java supports multiple interfaces).                        | Cannot extend another class (since Java supports single inheritance).                |
| Separation of task and execution | Runnable separates <b>task definition</b> from thread execution.                                          | Thread combines <b>task and thread execution</b> .                                   |
| Object creation                  | Only implements the task; needs a <code>Thread</code> object to start.                                    | Directly a thread; can call <code>start()</code> on it.                              |
| Resource overhead                | Slightly less overhead; better for creating <b>multiple threads sharing same task</b> .                   | Slightly more overhead; each Thread object is a thread itself.                       |

## Examples

### 1. Using Runnable

```
java

class MyTask implements Runnable {
    public void run() {
        System.out.println("Runnable running");
    }
}

public class Main {
    public static void main(String[] args) {
        Runnable task = new MyTask();
    }
}
```

```
Thread t = new Thread(task); // pass Runnable to Thread
t.start();
}
}
```

## 2. Using Thread

```
java

class MyThread extends Thread {
    public void run() {
        System.out.println("Thread running");
    }
}

public class Main {
    public static void main(String[] args) {
        Thread t = new MyThread();
        t.start();
    }
}
```

---

## Key Points

1. **Runnable** is preferred when your class already extends another class.
2. **Runnable** is better when you want multiple threads to share the same task.
3. **Thread** is simpler for quick, standalone thread creation.

---

If you want, I can also make a **tiny visual diagram showing the flow of Runnable vs Thread**, which makes this concept very easy to remember for interviews.

Do you want me to make it?

Explain Thread Pool and why it's better than creating threads manually.

Here's a detailed explanation of **Thread Pool** in Java and why it's better than creating threads manually:

---

## 1. What is a Thread Pool?

A **Thread Pool** is a collection of pre-instantiated reusable threads that are ready to execute tasks. Instead of creating a new thread for every task, tasks are submitted to the thread pool, which assigns them to available threads.

- Managed by `ExecutorService` in Java ( `java.util.concurrent` package).
- Threads are **reused**, which reduces the overhead of creating new threads.

### Example of creating a thread pool:

```
java

import java.util.concurrent.*;

public class ThreadPoolExample {
    public static void main(String[] args) {
        ExecutorService executor = Executors.newFixedThreadPool(3); // pool of 3 threads

        for (int i = 1; i <= 5; i++) {
            int taskId = i;
            executor.submit(() -> {
                System.out.println("Task " + taskId + " is running on " +
Thread.currentThread().getName());
            });
        }

        executor.shutdown(); // shutdown the pool
    }
}
```

### Output:

```
arduino

Task 1 is running on pool-1-thread-1
Task 2 is running on pool-1-thread-2
Task 3 is running on pool-1-thread-3
```

Task 4 is running on pool-1-thread-1

Task 5 is running on pool-1-thread-2

Notice how threads are reused for multiple tasks.

## 2. Why Thread Pool is Better than Creating Threads Manually

| Feature              | Manual Thread Creation                                                   | Thread Pool                                                          |
|----------------------|--------------------------------------------------------------------------|----------------------------------------------------------------------|
| Thread Creation Cost | High, each task creates a new thread.                                    | Low, threads are created once and reused.                            |
| Resource Management  | Difficult; too many threads may crash the system.                        | Efficient; thread number is fixed or managed automatically.          |
| Performance          | Slower for a large number of tasks due to frequent creation/destruction. | Faster; avoids repeated thread creation overhead.                    |
| Scalability          | Not scalable for large applications.                                     | Scales better; handles many tasks concurrently with limited threads. |
| Task Submission      | Requires manual thread start for each task.                              | Tasks are submitted to pool; scheduling handled automatically.       |
| Thread Reuse         | No                                                                       | Yes                                                                  |

## 3. Key Points

1. Use `Executors.newFixedThreadPool(n)` for a fixed number of threads.
2. Use `Executors.newCachedThreadPool()` for a dynamically growing pool.
3. Avoid manually creating thousands of threads; thread pool ensures **resource efficiency and better performance**.
4. Thread pools are widely used in **server applications, web servers, and concurrent processing**.

If you want, I can also make a **diagram showing task submission vs thread reuse in thread pool**, which makes it very intuitive for interviews.

Do you want me to make it?

Difference between synchronized block and synchronized method.

Here's a clear explanation of the **difference between** `synchronized` **block and** `synchronized` **method** in Java:

| Feature                  | Synchronized Method                                                                                           | Synchronized Block                                                                                                    |
|--------------------------|---------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| Definition               | The whole method is synchronized; only <b>one thread can execute it at a time</b> for the same object.        | Only a specific block of code inside a method is synchronized; the rest of the method can run concurrently.           |
| Lock Object              | Locks the <b>entire object</b> ( <code>this</code> ) for instance methods or <b>class</b> for static methods. | You can specify <b>any object</b> as a lock ( <code>synchronized(lockObj)</code> ), giving more fine-grained control. |
| Scope of Synchronization | Entire method body                                                                                            | Only the block enclosed by <code>synchronized</code>                                                                  |
| Performance              | Less efficient if the method has a large body; even non-critical code is blocked.                             | More efficient; only critical section is locked.                                                                      |
| Usage                    | Simple and easy to use when the whole method needs thread safety.                                             | Useful when <b>only part of the method</b> needs synchronization or different locks are needed.                       |

## Examples

### 1. Synchronized Method

java

```
class Counter {  
    private int count = 0;  
  
    public synchronized void increment() {  
        count++; // entire method synchronized  
    }  
  
    public int getCount() {  
        return count;  
    }  
}
```

## 2. Synchronized Block

java

```
class Counter {  
    private int count = 0;  
    private final Object lock = new Object();  
  
    public void increment() {  
        synchronized(lock) { // only this block is synchronized  
            count++;  
        }  
        // Other code here can run concurrently  
        System.out.println("Increment done");  
    }  
  
    public int getCount() {  
        return count;  
    }  
}
```

---

## Key Points

1. Use **synchronized method** for simplicity when **whole method** needs thread safety.
2. Use **synchronized block** for **better performance**, **fine-grained locking**, or when you want to lock **different objects**.

- Both prevent **race conditions** by allowing only one thread to access the critical section at a time.

If you want, I can also make a **tiny diagram showing how synchronized method locks the whole object vs synchronized block locking only a part**, which is very helpful for interviews.

Do you want me to make it?

Difference between composition and inheritance.

Here's a clear comparison between **composition** and **inheritance** in Java:

| Feature                  | Inheritance                                                       | Composition                                                                       |
|--------------------------|-------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| Definition               | Represents an “ <b>is-a</b> ” relationship between classes.       | Represents a “ <b>has-a</b> ” relationship between classes.                       |
| Code Reuse               | Achieved by <b>extending</b> a parent class.                      | Achieved by <b>including objects</b> of other classes as fields.                  |
| Tight Coupling           | Creates <b>tight coupling</b> between parent and child classes.   | Creates <b>looser coupling</b> , easier to change or replace components.          |
| Flexibility              | Less flexible; hard to switch base class at runtime.              | More flexible; components can be changed at runtime.                              |
| Access to Parent Members | Child class can access parent's <b>public/protected members</b> . | No direct access; methods of the included object must be called explicitly.       |
| Multiple Inheritance     | Not allowed in Java for classes (only via interfaces).            | Can include multiple objects of different types, simulating multiple inheritance. |

| Feature     | Inheritance                                              | Composition                                                                    |
|-------------|----------------------------------------------------------|--------------------------------------------------------------------------------|
| When to Use | When there is a <b>clear hierarchical relationship</b> . | When you want to <b>reuse functionality</b> without forming a rigid hierarchy. |

## Examples

### 1. Inheritance (is-a relationship)

```
java

class Vehicle {
    void start() { System.out.println("Vehicle started"); }
}

class Car extends Vehicle { // Car "is-a" Vehicle
    void honk() { System.out.println("Car honks"); }
}

public class Main {
    public static void main(String[] args) {
        Car c = new Car();
        c.start(); // inherited from Vehicle
        c.honk();
    }
}
```

### 2. Composition (has-a relationship)

```
java

class Engine {
    void start() { System.out.println("Engine started"); }
}

class Car {
    private Engine engine; // Car "has-a" Engine

    Car() { this.engine = new Engine(); }
}
```

```
void startCar() {  
    engine.start(); // delegate to Engine  
    System.out.println("Car started");  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Car c = new Car();  
        c.startCar();  
    }  
}
```

---

## Key Points

1. **Inheritance** = "is-a" → forms hierarchy, less flexible.
2. **Composition** = "has-a" → uses objects of other classes, more flexible and loosely coupled.
3. Modern Java design favors **composition over inheritance** for better maintainability.

---

If you want, I can make a **tiny diagram showing "is-a" vs "has-a" relationships**—it's very helpful for interviews.

Do you want me to make it?

What is locking? Difference between pessimistic and optimistic locking.

Here's a clear explanation of **locking in databases** and the difference between **pessimistic** and **optimistic locking**:

---

## 1. What is Locking?

**Locking** is a mechanism used in DBMS to **control concurrent access to data** so that **multiple transactions** do not cause inconsistencies.

- Ensures **data integrity** and **consistency** during **concurrent operations**.
- Prevents problems like **lost updates**, **dirty reads**, and **phantom reads**.

Types of locks:

1. **Shared Lock (S-lock / Read lock)** → Multiple transactions can read, but cannot write.
2. **Exclusive Lock (X-lock / Write lock)** → Only one transaction can read/write; others are blocked.

## 2. Pessimistic vs Optimistic Locking

| Feature                   | Pessimistic Locking                                                                       | Optimistic Locking                                                                    |
|---------------------------|-------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| <b>Assumption</b>         | Conflicts are <b>likely</b> ; lock data before access.                                    | Conflicts are <b>rare</b> ; allow access and check before commit.                     |
| <b>Locking</b>            | Locks the data immediately (read or write) until transaction completes.                   | Does not lock data; uses <b>versioning or timestamps</b> to detect changes at commit. |
| <b>Performance</b>        | Can <b>reduce concurrency</b> ; may cause blocking.                                       | Higher concurrency; better performance for low-conflict scenarios.                    |
| <b>Conflict Detection</b> | Prevents conflicts proactively.                                                           | Detects conflicts <b>after transaction</b> completes.                                 |
| <b>Use Case</b>           | High-contention environments where multiple transactions frequently update the same data. | Low-contention environments where conflicts are rare.                                 |

### Example 1: Pessimistic Locking (SQL)

```
sql

-- Lock a row for update
SELECT * FROM account
WHERE id = 1
FOR UPDATE; -- row is locked until transaction completes
```

```
UPDATE account SET balance = balance - 100 WHERE id = 1;
COMMIT;
```

- Other transactions trying to update the same row will **wait**.

---

## Example 2: Optimistic Locking (Java/JPA)

```
java

@Entity
class Account {
    @Id
    private Long id;

    private int balance;

    @Version      // version field for optimistic locking
    private int version;
}

// Transaction 1 and 2 read same row concurrently
// Update will fail if version has changed during commit
```

- Transaction will **fail** if the row was modified by another transaction; it can retry.

---

## Key Points

1. **Pessimistic Locking:** Safe, blocks others → lower concurrency.
2. **Optimistic Locking:** Non-blocking, detects conflicts → higher concurrency.
3. **Use pessimistic** in high-conflict scenarios; **use optimistic** in low-conflict, read-heavy scenarios.

---

If you want, I can also make a **tiny diagram showing pessimistic vs optimistic locking workflow**, which makes it very easy to visualize for interviews.

Do you want me to make it?

Explain foreign key constraint cascade options (ON DELETE CASCADE, ON UPDATE CASCADE).

Here's a clear explanation of **foreign key cascade options** in SQL:

## 1. What is a Foreign Key?

A **foreign key** is a column (or set of columns) in one table that **refers to the primary key** of another table.

- Ensures **referential integrity** between tables.
- Prevents invalid or orphaned records.

## 2. Cascade Options

Cascade options define **what happens to child table rows** when a **parent row is updated or deleted**.

### a) ON DELETE CASCADE

- If a row in the **parent table** is **deleted**, all **related rows in the child table** are also **automatically deleted**.
- Prevents orphaned child records.

**Example:**

sql

```
CREATE TABLE Department (  
  dept_id INT PRIMARY KEY,  
  name VARCHAR(50)  
);  
  
CREATE TABLE Employee (  
  emp_id INT PRIMARY KEY,  
  name VARCHAR(50),  
  dept_id INT,  
  FOREIGN KEY (dept_id) REFERENCES Department(dept_id)  
  ON DELETE CASCADE  
);
```

```
-- Delete department with id 1
```

```
DELETE FROM Department WHERE dept_id = 1;
```

```
-- All employees in that department are also deleted automatically
```

---

## b) ON UPDATE CASCADE

- If the **primary key of the parent table** is **updated**, the **foreign keys in child table** are also updated automatically.
- Maintains consistency when parent keys change.

### Example:

```
sql
```

```
CREATE TABLE Employee (  
  emp_id INT PRIMARY KEY,  
  name VARCHAR(50),  
  dept_id INT,  
  FOREIGN KEY (dept_id) REFERENCES Department(dept_id)  
  ON UPDATE CASCADE  
);
```

```
-- Update department id from 1 → 10
```

```
UPDATE Department SET dept_id = 10 WHERE dept_id = 1;
```

```
-- All employees with dept_id = 1 now automatically have dept_id = 10
```

---

## Key Points

1. **ON DELETE CASCADE** → Child rows deleted automatically when parent is deleted.
2. **ON UPDATE CASCADE** → Child foreign keys updated automatically when parent key changes.
3. Cascade options **ensure referential integrity** and reduce manual updates.
4. Use carefully; deleting a parent row can **delete a lot of child data** unintentionally.

---

If you want, I can make a **tiny diagram showing parent-child relationship with ON DELETE and ON UPDATE cascade**, which makes it very easy to visualize for interviews.

Do you want me to make it?

Here's a clear comparison between **OLTP** and **OLAP** systems:

| Feature       | OLTP (Online Transaction Processing)                                      | OLAP (Online Analytical Processing)                                              |
|---------------|---------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| Purpose       | Manages <b>day-to-day transactional data</b> (inserts, updates, deletes). | Supports <b>analytical queries and decision-making</b> (reporting, data mining). |
| Data Type     | Current, detailed, operational data.                                      | Historical, aggregated, summarized data.                                         |
| Transactions  | Many <b>short, simple, frequent transactions</b> .                        | Fewer, <b>complex, long-running queries</b> .                                    |
| Normalization | Highly normalized to reduce redundancy.                                   | Often denormalized for faster query performance.                                 |
| Response Time | Fast response required (milliseconds).                                    | Slower response acceptable (seconds to minutes) due to heavy queries.            |
| Users         | Clerks, bank tellers, front-office staff.                                 | Managers, analysts, decision-makers.                                             |
| Examples      | Banking transactions, online shopping orders, airline reservations.       | Business intelligence reports, trend analysis, data warehouses.                  |
| Data Volume   | Smaller, operational datasets.                                            | Large volumes of historical data.                                                |

## Key Points

1. **OLTP** = Transaction-focused, supports routine operations.
2. **OLAP** = Analysis-focused, supports complex queries and decision-making.

3. **OLTP** is fast, normalized, and handles many concurrent users.
4. **OLAP** is slower, optimized for read-heavy queries, and often uses star or snowflake schema.

If you want, I can also make a **diagram showing OLTP vs OLAP workflow**, which makes it easier to remember for interviews.

Do you want me to make it?

Explain database normalization vs denormalization with examples.

Here's a clear explanation of **database normalization vs denormalization** in DBMS:

## 1. Database Normalization

**Normalization** is the process of organizing database tables to **reduce redundancy** and **improve data integrity**.

- Splits large tables into smaller related tables.
- Ensures each piece of data is stored **once**.
- Follows **Normal Forms (1NF, 2NF, 3NF, BCNF, etc.)**.

### Example of Normalization

#### Unnormalized Table (Student + Course + Marks)

| StudentID | StudentName | CourseID | CourseName | Marks |
|-----------|-------------|----------|------------|-------|
| 1         | Alice       | C1       | Math       | 95    |
| 1         | Alice       | C2       | Physics    | 90    |
| 2         | Bob         | C1       | Math       | 85    |

**Problems:** Redundant data ( **Alice** repeated), update anomalies.

#### Normalized Tables (3NF):

### 1. Student Table

```
| StudentID | StudentName |
|-----|-----|
| 1 | Alice |
| 2 | Bob |
```

### 2. Course Table

```
| CourseID | CourseName |
|-----|-----|
| C1 | Math |
| C2 | Physics |
```

### 3. Marks Table

```
| StudentID | CourseID | Marks |
|-----|-----|-----|
| 1 | C1 | 95 |
| 1 | C2 | 90 |
| 2 | C1 | 85 |
```

✅ Benefits: Reduced redundancy, easy updates, high data integrity.

---

## 2. Database Denormalization

**Denormalization** is the process of **combining tables** to reduce **complex joins** and improve **query performance**.

- Introduces some redundancy intentionally.
- Common in **OLAP** and reporting systems.

### Example of Denormalization

Instead of splitting into multiple tables, you might keep a **single table**:

| StudentID | StudentName | CourseID | CourseName | Marks |
|-----------|-------------|----------|------------|-------|
| 1         | Alice       | C1       | Math       | 95    |
| 1         | Alice       | C2       | Physics    | 90    |
| 2         | Bob         | C1       | Math       | 85    |

✅ Benefits: Faster **read-heavy queries**, fewer joins.

⚠️ Drawbacks: More storage, update anomalies possible.

---

## Key Differences

| Feature         | Normalization                                | Denormalization               |
|-----------------|----------------------------------------------|-------------------------------|
| Purpose         | Reduce redundancy & maintain integrity       | Improve query performance     |
| Data Redundancy | Eliminated                                   | Introduced intentionally      |
| Schema          | Many small tables                            | Fewer large tables            |
| Performance     | Slower for read-heavy queries (due to joins) | Faster for read-heavy queries |
| Use Case        | OLTP systems                                 | OLAP, reporting systems       |

---

If you want, I can also make a **diagram showing normalized vs denormalized table structure**, which is very handy for interviews.

Do you want me to make it?

How does GROUP BY work?

Great question 🙌 Let's break down **how GROUP BY works in SQL** in a simple and interview-friendly way:

---

### 1. What is GROUP BY?

- `GROUP BY` is used to **group rows that have the same values in specified columns**.
- After grouping, you can apply **aggregate functions** ( `COUNT` , `SUM` , `AVG` , `MIN` , `MAX` ) to each group.

---

### 2. How it Works (Step-by-Step)

1. **FROM & WHERE** → First, SQL selects rows from the table (with filtering if **WHERE** is used).
  2. **GROUP BY** → Then, it groups rows that have the same column values together.
  3. **Aggregate Functions** → Applied to each group (e.g., count number of rows, calculate totals).
  4. **HAVING** → Filters groups based on aggregate conditions (similar to **WHERE** , but for groups).
  5. **ORDER BY** → Sorts the final grouped result.
- 

### 3. Example

**Table:** **Orders**

| OrderID | Customer | Amount |
|---------|----------|--------|
| 1       | Alice    | 100    |
| 2       | Bob      | 200    |
| 3       | Alice    | 150    |
| 4       | Bob      | 50     |
| 5       | Alice    | 200    |

---

#### Query:

sql

```
SELECT Customer, SUM(Amount) AS TotalAmount
FROM Orders
GROUP BY Customer;
```

---

#### Result:

| Customer | TotalAmount |
|----------|-------------|
| Alice    | 450         |
| Bob      | 250         |

✅ SQL grouped all rows by `Customer` and then summed the `Amount` for each group.

## 4. Using HAVING

If we only want customers who spent more than 300:

sql

```
SELECT Customer, SUM(Amount) AS TotalAmount
FROM Orders
GROUP BY Customer
HAVING SUM(Amount) > 300;
```

**Result:**

| Customer | TotalAmount |
|----------|-------------|
| Alice    | 450         |

## 5. Key Points to Remember

- `WHERE` filters rows before grouping, `HAVING` filters groups after grouping.
- Columns in the `SELECT` must be either:
  1. Part of the `GROUP BY` clause
  2. Or used inside an aggregate function.
- Often used in **reporting, analytics, and summaries**.

👉 Would you like me to also give you some **common tricky interview questions on `GROUP BY` with answers**, so you're ready for follow-ups?

Difference between inner and outer transactions. and

Good one 👍 Let's carefully go through **inner and outer transactions** in databases (often asked in intermediate-level DBMS interviews):

---

## 1. What is a Transaction?

A **transaction** is a logical unit of work in a database (series of SQL statements) that must be **executed completely or not at all** (ACID properties).

Example: Transfer money → deduct from one account, add to another.

---

## 2. Inner vs Outer Transactions

When **nested transactions** exist (a transaction inside another), we distinguish between **outer** and **inner** transactions.

### Outer Transaction

- The **main (parent) transaction** started explicitly by the user/application.
- Controls the **overall commit or rollback** of all nested transactions.

### Inner Transaction

- A **nested (child) transaction** started inside the outer transaction.
  - Executes independently but its final outcome **depends on the outer transaction's fate**.
- 

## 3. Example

sql

```
BEGIN TRANSACTION OuterTransaction;
```

```
UPDATE Accounts SET balance = balance - 500 WHERE acc_id = 1;
```

```
BEGIN TRANSACTION InnerTransaction;
```

```
    UPDATE Accounts SET balance = balance + 500 WHERE acc_id = 2;
```

```
COMMIT TRANSACTION InnerTransaction;
```

```
-- OuterTransaction still not committed yet
ROLLBACK TRANSACTION OuterTransaction;
```

## What Happens?

- Even though `InnerTransaction` was committed,
- Since the **OuterTransaction** rolled back,
- **Both updates are undone.**

✓ **Rule:** The final outcome of all inner transactions depends on the outer transaction.

## 4. Key Differences

| Aspect          | Outer Transaction                                                              | Inner Transaction                                                              |
|-----------------|--------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| Definition      | The main transaction initiated by the user.                                    | A nested transaction inside an outer transaction.                              |
| Control         | Controls the final commit/rollback.                                            | Executes independently but outcome depends on the outer one.                   |
| Rollback Impact | Rolling back outer undoes everything (including committed inner transactions). | Rolling back inner does not affect outer, but outer can still succeed or fail. |
| Use Case        | High-level logical operation.                                                  | Breaking down complex tasks into sub-steps.                                    |

## 5. Where Used?

- **Databases that support nested transactions** (e.g., Oracle, SQL Server partially with savepoints).
- Useful in **modular systems**: each module can execute its own transaction logic but still depend on the overall operation's success.

👉 Do you want me to also explain **SAVEPOINT vs Inner Transaction**, since interviewers often ask this as a follow-up?

clustered and non-clustered index.

Great question 🔥 — **Indexes** are very important in DBMS interviews. Let's break down **Clustered vs Non-Clustered Index** in a simple way:

---

## 1. What is an Index?

An **index** is like a **book index**:

- Speeds up searching in a database.
  - Instead of scanning the whole table, DB engine uses the index to quickly locate data.
- 

## 2. Clustered Index

- **Data is physically stored** in the same order as the index.
- Each table can have **only one clustered index** (because rows can only be sorted one way).
- By default, the **Primary Key** creates a clustered index (in most RDBMS like SQL Server, MySQL InnoDB).

📌 Example:

If you create a clustered index on `StudentID`, the table's rows are physically arranged in ascending `StudentID`.

---

## 3. Non-Clustered Index

- **Separate structure from the table** → index has pointers (row locators) to the actual data rows.
- Does **not change the physical order** of data.
- A table can have **multiple non-clustered indexes** (like multiple “search shortcuts”).

📌 Example:

If you create a non-clustered index on `StudentName`, the DBMS creates a separate structure with `StudentName` + a pointer to the actual row in the clustered index/table.

---

## 4. Key Differences

| Feature          | Clustered Index                                             | Non-Clustered Index                          |
|------------------|-------------------------------------------------------------|----------------------------------------------|
| Physical storage | Data rows are stored in the index order.                    | Index stores pointers to data rows.          |
| Number per table | Only one allowed.                                           | Multiple allowed.                            |
| Default          | Created automatically on <b>Primary Key</b> .               | Must be created explicitly.                  |
| Speed            | Faster for range queries ( <code>BETWEEN</code> , sorting). | Slightly slower (extra lookup needed).       |
| Size             | Occupies less space (data + index together).                | Requires extra storage (separate structure). |

---

## 5. Visual Analogy

- **Clustered Index** = Like a **phonebook** sorted by name → data itself is in order.
  - **Non-Clustered Index** = Like the **index section at the back of a book** → points to pages where data exists.
- 

## 6. Example in SQL

```
sql

-- Clustered Index (usually primary key)
CREATE CLUSTERED INDEX idx_student_id
ON Students(StudentID);

-- Non-Clustered Index
CREATE NONCLUSTERED INDEX idx_student_name
ON Students(StudentName);
```

---

✓ **Summary:**

- Use **clustered index** for columns often used in **range queries and sorting**.
- Use **non-clustered indexes** for **search columns** (e.g., `WHERE StudentName = 'Alice'`).

Would you like me to also give you a **real-world example with query performance difference** (with and without index) so you can explain better in interviews?

sieve of eratosthenes  
upto 100

Got it 👍 You want the **Sieve of Eratosthenes** algorithm to find all prime numbers up to **100**.

## 1. Idea of the Algorithm

- Create a boolean array `prime[0..n]` and initialize all entries as true.
- Starting from the first prime `2`, mark all its multiples as **not prime**.
- Repeat for the next unmarked number.
- Continue until  $\sqrt{n}$ .
- Remaining unmarked numbers are primes.

## 2. Java Implementation (Up to 100)

java

```
public class SieveOfEratosthenes {  
    public static void main(String[] args) {  
        int n = 100;  
        boolean[] isPrime = new boolean[n + 1];  
  
        // Assume all numbers are prime initially  
        for (int i = 2; i <= n; i++) {  
            isPrime[i] = true;  
        }  
  
        // Mark non-primes
```

```

for (int p = 2; p * p <= n; p++) {
    if (isPrime[p]) {
        for (int multiple = p * p; multiple <= n; multiple += p) {
            isPrime[multiple] = false;
        }
    }
}

// Print primes
System.out.println("Prime numbers up to " + n + ":");
for (int i = 2; i <= n; i++) {
    if (isPrime[i]) {
        System.out.print(i + " ");
    }
}
}
}

```

### 3. Output

CSS

Prime numbers up to 100:

2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89 97

✓ **Time Complexity:**  $O(n \log \log n)$

✓ **Space Complexity:**  $O(n)$

Would you like me to also give you the **step-by-step table of how numbers get marked as composite** (like in classroom explanations), so you can visualize the sieve process for interviews?

fibonacci series using tabulation and memoization

Perfect question 🧠 Fibonacci is the **classic DP problem**. Let's see both **memoization (top-down)** and **tabulation (bottom-up)** approaches in **Java**:

---

## 1. Fibonacci with Memoization (Top-Down)

👉 Store results in an array (cache) to **avoid recomputation**.

```
java

import java.util.Arrays;

public class FibonacciMemoization {
    static int[] dp;

    public static int fib(int n) {
        // Base cases
        if (n <= 1) return n;

        // If already computed, return it
        if (dp[n] != -1) return dp[n];

        // Store result in dp array
        dp[n] = fib(n - 1) + fib(n - 2);
        return dp[n];
    }

    public static void main(String[] args) {
        int n = 10;
        dp = new int[n + 1];
        Arrays.fill(dp, -1);

        System.out.println("Fibonacci using Memoization:");
        for (int i = 0; i <= n; i++) {
            System.out.print(fib(i) + " ");
        }
    }
}
```

✅ **Output:**

```
cpp
```

Fibonacci using Memoization:

0 1 1 2 3 5 8 13 21 34 55

## 2. Fibonacci with Tabulation (Bottom-Up)

👉 Build solution from the **ground up** using iteration.

java

```
public class FibonacciTabulation {
    public static void main(String[] args) {
        int n = 10;
        int[] dp = new int[n + 1];

        // Base cases
        dp[0] = 0;
        dp[1] = 1;

        // Bottom-up fill
        for (int i = 2; i <= n; i++) {
            dp[i] = dp[i - 1] + dp[i - 2];
        }

        System.out.println("Fibonacci using Tabulation:");
        for (int i = 0; i <= n; i++) {
            System.out.print(dp[i] + " ");
        }
    }
}
```

✅ **Output:**

cpp

Fibonacci using Tabulation:

0 1 1 2 3 5 8 13 21 34 55

## 3. Comparison

| Feature  | Memoization (Top-Down)                          | Tabulation (Bottom-Up)                                    |
|----------|-------------------------------------------------|-----------------------------------------------------------|
| Approach | Recursive + caching                             | Iterative                                                 |
| Calls    | Starts with <code>fib(n)</code> and breaks down | Builds from <code>fib(0)</code> up to <code>fib(n)</code> |
| Overhead | Function call stack                             | Simple loop                                               |
| Space    | <code>O(n)</code> (cache + recursion stack)     | <code>O(n)</code> (array only)                            |
| Best for | When subproblems are not always needed          | When all subproblems are needed                           |

👉 Quick Tip for interviews: You can further **optimize space** in tabulation to  **$O(1)$**  by only storing the last two values instead of the full array.

Would you like me to also show that **space-optimized Fibonacci** version (iterative with just two variables)?

tell me more question like pallindrome armstrong  
fibonachi9 question that cna be there in interview

Got it 👍 You're asking about **classic coding/interview questions** (like Palindrome, Armstrong, Fibonacci) that interviewers often ask to test **logic + basics** in Java (or any language).

Here's a list grouped by category:

## 1. Number-based Questions

- ✅ **Palindrome Number** → Check if a number reads the same forward and backward.

- **✓ Armstrong Number** → Sum of each digit raised to the power of total digits = number.
  - **✓ Fibonacci Series** → Print series, nth Fibonacci, or using recursion/DP.
  - **✓ Prime Number** → Check if a number is prime, generate primes up to N (Sieve).
  - **✓ Factorial** → Find factorial using recursion and iteration.
  - **✓ Reverse a Number** → Reverse digits of an integer.
  - **✓ Sum of Digits** → Repeated sum until single digit (digital root).
  - **✓ Perfect Number** → Sum of divisors = number.
  - **✓ Strong Number** → Sum of factorial of digits = number.
  - **✓ Harshad Number** → Divisible by the sum of its digits.
- 

## 2. String-based Questions

- **✓ Palindrome String** → Check if a string reads the same backward.
  - **✓ Anagram** → Check if two strings contain the same characters.
  - **✓ String Reversal** → Without using built-in functions.
  - **✓ Count Vowels & Consonants.**
  - **✓ First Non-Repeated Character** in a string.
  - **✓ Check Pangram** → Contains all alphabets A–Z.
  - **✓ Check Isogram** → No repeating characters.
  - **✓ Permutations of a String** (backtracking).
- 

## 3. Array-based Questions

- **✓ Find Largest & Smallest Element.**
  - **✓ Second Largest/Smallest element.**
  - **✓ Reverse an Array.**
  - **✓ Check if Array is Sorted.**
  - **✓ Find Missing Number** in 1..n sequence.
  - **✓ Find Duplicate Elements.**
  - **✓ Rotate Array** (left/right rotation).
  - **✓ Binary Search / Linear Search.**
  - **✓ Matrix Programs** → Transpose, Spiral print, Diagonal sum.
- 

## 4. Pattern Printing Questions

- ☒ Pyramid, Inverted Pyramid.
  - ☒ Pascal's Triangle.
  - ☒ Floyd's Triangle.
  - ☒ Diamond pattern with \* or numbers.
- 

## 5. Logical / Miscellaneous

- ☒ Swap two numbers without a third variable.
  - ☒ Check Leap Year.
  - ☒ LCM and GCD of two numbers.
  - ☒ Convert Decimal ↔ Binary / Octal / Hex.
  - ☒ Count digits / Count frequency of each digit.
  - ☒ Check if number is Power of 2.
  - ☒ Generate Random Numbers (Math.random / Random class).
- 

### 🔥 Tip for Interviews:

Most interviewers don't care about huge complex solutions here. They check:

- ☒ Can you solve it logically?
  - ☒ Do you know both **iterative and recursive** approaches?
  - ☒ Can you handle **edge cases** (like negative numbers, empty strings, large inputs)?
- 

👉 Do you want me to prepare a **practice set of 15–20 most common coding questions with short sample Java code** so you can revise them quickly before interviews?

Check if number is Power of 2.  
Generate Random Numbers (Math.random / Random class).  
. Pattern Printing Questions

☒ Pyramid, Inverted Pyramid.

☒ Pascal's Triangle.

✓ Floyd's Triangle.

✓ Diamond pattern with \* or numbers.

write program for all of these

Sure! Let's go **step by step** and write **Java programs** for all the tasks you mentioned. I'll provide **clean and simple code** suitable for interviews.

---

## 1. Check if a Number is Power of 2

java

```
public class PowerOfTwo {
    public static boolean isPowerOfTwo(int n) {
        if (n <= 0) return false;
        return (n & (n - 1)) == 0; // Bitwise trick
    }

    public static void main(String[] args) {
        int num = 16;
        System.out.println(num + " is power of 2? " + isPowerOfTwo(num));
    }
}
```

## 2. Generate Random Numbers

### a) Using Math.random()

java

```
public class RandomNumberMath {
    public static void main(String[] args) {
        int min = 1, max = 100;
        int randomNum = (int)(Math.random() * (max - min + 1)) + min;
        System.out.println("Random Number (Math.random): " + randomNum);
    }
}
```

## b) Using Random Class

```
java

import java.util.Random;

public class RandomNumberClass {
    public static void main(String[] args) {
        Random rand = new Random();
        int randomNum = rand.nextInt(100) + 1; // 1 to 100
        System.out.println("Random Number (Random class): " + randomNum);
    }
}
```

---

## 3. Pyramid Pattern

```
java

public class PyramidPattern {
    public static void main(String[] args) {
        int rows = 5;
        for (int i = 1; i <= rows; i++) {
            for (int j = rows; j > i; j--) System.out.print(" ");
            for (int k = 1; k <= (2*i - 1); k++) System.out.print("*");
            System.out.println();
        }
    }
}
```

**Output (5 rows):**

```
markdown

*
***
*****
*****
*****
```

---

## 4. Inverted Pyramid

java

```
public class InvertedPyramid {  
    public static void main(String[] args) {  
        int rows = 5;  
        for (int i = rows; i >= 1; i--) {  
            for (int j = 1; j <= rows - i; j++) System.out.print(" ");  
            for (int k = 1; k <= (2*i - 1); k++) System.out.print("*");  
            System.out.println();  
        }  
    }  
}
```

## 5. Pascal's Triangle

java

```
public class PascalsTriangle {  
    public static void main(String[] args) {  
        int rows = 5;  
        for (int i = 0; i < rows; i++) {  
            int number = 1;  
            for (int j = 0; j <= i; j++) {  
                System.out.print(number + " ");  
                number = number * (i - j) / (j + 1); // Binomial coefficient  
            }  
            System.out.println();  
        }  
    }  
}
```

Output (5 rows):

```
1  
1 1  
1 2 1
```

```
1 3 3 1
1 4 6 4 1
```

---

## 6. Floyd's Triangle

java

```
public class FloydsTriangle {
    public static void main(String[] args) {
        int rows = 5, num = 1;
        for (int i = 1; i <= rows; i++) {
            for (int j = 1; j <= i; j++) {
                System.out.print(num++ + " ");
            }
            System.out.println();
        }
    }
}
```

**Output (5 rows):**

```
1
2 3
4 5 6
7 8 9 10
11 12 13 14 15
```

---

## 7. Diamond Pattern

java

```
public class DiamondPattern {
    public static void main(String[] args) {
        int rows = 5;

        // Upper Pyramid
        for (int i = 1; i <= rows; i++) {
```

```

        for (int j = rows; j > i; j--) System.out.print(" ");
        for (int k = 1; k <= (2*i - 1); k++) System.out.print("x");
        System.out.println();
    }

    // Lower Pyramid
    for (int i = rows - 1; i >= 1; i--) {
        for (int j = 1; j <= rows - i; j++) System.out.print(" ");
        for (int k = 1; k <= (2*i - 1); k++) System.out.print("x");
        System.out.println();
    }
}
}

```

**Output (5 rows upper + 4 rows lower):**

markdown

```

      *
    ***
  *****
*****
*****
*****
*****
  *****
    ***
      *

```

---

✅ These programs cover **all basic patterns, number checks, and random generation** that are commonly asked in **Java interviews**.

---

If you want, I can also make a **single file with all of these programs together** in a **menu-driven format**, so you can **run and test any pattern or number question easily**.

Do you want me to do that?

Got it 👍 You're asking about **more complex pattern problems + Pascal's triangle variations**, which are **very common in Java interviews**. Let me give you a list with **Java solutions**.

---

## ◆ 1. Pascal's Triangle – Standard

java

```
public class PascalTriangle {
    public static void main(String[] args) {
        int rows = 6;

        for (int i = 0; i < rows; i++) {
            // Print spaces for alignment
            for (int s = 0; s < rows - i; s++) {
                System.out.print(" ");
            }
            int num = 1;
            for (int j = 0; j <= i; j++) {
                System.out.print(num + " ");
                num = num * (i - j) / (j + 1); // nCr formula
            }
            System.out.println();
        }
    }
}
```

✓ Output:

markdown

```
1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
1 5 10 10 5 1
```

## ◆ 2. Pascal's Triangle (Only Odd/Even Values as Pattern)

👉 Print only **odd numbers** as **1** and ignore even.

java

```
public class PascalOddPattern {  
    public static void main(String[] args) {  
        int rows = 7;  
  
        for (int i = 0; i < rows; i++) {  
            for (int s = 0; s < rows - i; s++) {  
                System.out.print(" ");  
            }  
            int num = 1;  
            for (int j = 0; j <= i; j++) {  
                if (num % 2 == 1)  
                    System.out.print("1 ");  
                else  
                    System.out.print(" ");  
                num = num * (i - j) / (j + 1);  
            }  
            System.out.println();  
        }  
    }  
}
```

✅ Output:

markdown

```
1  
1 1  
1 1  
1 1 1 1  
1 1  
1 1 1 1  
1 1 1 1
```

### ◆ 3. Floyd's Triangle (Numbers in Sequence)

java

```
public class FloydsTriangle {  
    public static void main(String[] args) {  
        int rows = 6, num = 1;  
        for (int i = 1; i <= rows; i++) {  
            for (int j = 1; j <= i; j++) {  
                System.out.print(num++ + " ");  
            }  
            System.out.println();  
        }  
    }  
}
```

✓ Output:

```
1  
2 3  
4 5 6  
7 8 9 10  
11 12 13 14 15  
16 17 18 19 20 21
```

---

### ◆ 4. Diamond Pattern with Stars

java

```
public class DiamondPattern {  
    public static void main(String[] args) {  
        int n = 5;  
  
        // Upper half  
        for (int i = 1; i <= n; i++) {  
            for (int s = 1; s <= n - i; s++) {  
                System.out.print(" ");  
            }  
            for (int j = 1; j <= 2 * i - 1; j++) {
```

```

        System.out.print(" *");
    }
    System.out.println();
}

// Lower half
for (int i = n - 1; i >= 1; i--) {
    for (int s = 1; s <= n - i; s++) {
        System.out.print(" ");
    }
    for (int j = 1; j <= 2 * i - 1; j++) {
        System.out.print(" *");
    }
    System.out.println();
}
}
}

```

✓ Output:

markdown

```

 *
***
*****
*****
*****
*****
*****
*****
***
 *

```

## ◆ 5. Hollow Diamond with Stars

java

```

public class HollowDiamond {
    public static void main(String[] args) {
        int n = 5;
    }
}

```

```

// Upper half
for (int i = 1; i <= n; i++) {
    for (int s = 1; s <= n - i; s++) {
        System.out.print(" ");
    }
    for (int j = 1; j <= 2 * i - 1; j++) {
        if (j == 1 || j == 2 * i - 1)
            System.out.print("*");
        else
            System.out.print(" ");
    }
    System.out.println();
}

// Lower half
for (int i = n - 1; i >= 1; i--) {
    for (int s = 1; s <= n - i; s++) {
        System.out.print(" ");
    }
    for (int j = 1; j <= 2 * i - 1; j++) {
        if (j == 1 || j == 2 * i - 1)
            System.out.print("*");
        else
            System.out.print(" ");
    }
    System.out.println();
}
}
}

```

✓ Output:

markdown

```

  *
 * *
*   *
*   *
*   *
*   *
*   *

```

\* \*

\*

## ◆ 6. Pascal's Triangle – Sum of Row Elements

👉 A twist often asked in interviews: The **sum of the nth row** =  $2^n$ .

We can print **row sums** using Pascal.

java

```
public class PascalRowSum {  
    public static void main(String[] args) {  
        int rows = 6;  
  
        for (int i = 0; i < rows; i++) {  
            System.out.println("Row " + i + " sum = " + (1 << i)); // 2^i  
        }  
    }  
}
```

✅ Output:

bash

```
Row 0 sum = 1  
Row 1 sum = 2  
Row 2 sum = 4  
Row 3 sum = 8  
Row 4 sum = 16  
Row 5 sum = 32
```

👉 These are **medium–hard level pattern + Pascal questions** that interviewers love.

Would you like me to also add **number-based complex patterns** (like **palindromic number pyramid**, **zigzag numbers**, etc.), which are asked in coding rounds of TCS / Infosys / Capgemini?

Got it 👍 Since you're preparing for **Capgemini interview (Analyst/Exceller program)**, I'll break it into **two parts**:

---

## ◆ 1. Capgemini Interview – Coding Questions

In recent drives, Capgemini's **Coding Round** includes **2 questions** (medium level) to solve in 30–45 mins.

Here are the **popular coding problems** asked:

### ✓ Pattern & Math Problems

- Print **Fibonacci Series / Nth Fibonacci** (DP expected).
  - Check if a number is **Armstrong / Palindrome / Strong / Perfect / Prime**.
  - Generate **Pascal's Triangle / Floyd's Triangle / Pyramid patterns**.
  - Check if a number is **Power of 2**.
  - Find **GCD / LCM** of two numbers.
  - Print **Diamond / Hollow Pyramid / Zigzag pattern**.
- 

### ✓ String Problems

- Check if a string is **Palindrome**.
  - Check if two strings are **Anagrams**.
  - Count **vowels, consonants, digits, and spaces**.
  - Remove **duplicate characters**.
  - Print **first non-repeating character**.
  - Reverse words in a sentence.
- 

### ✓ Array Problems

- Find **second largest / smallest element**.
- Print **leaders in an array**.
- Rotate array (left/right rotation).
- Find **missing number** in 1 to N.
- Find **pair of numbers with given sum**.
- Merge and sort two arrays.

---

## ✓ DBMS / SQL Queries

- Find **second highest salary**.
  - Write query for **employees earning above average salary**.
  - Difference between `DELETE` , `TRUNCATE` , and `DROP` .
  - Explain **Normalization vs Denormalization** with example.
  - Write query to find **duplicate records** in a table.
- 

## ♦ 2. Capgemini Interview – Other Questions

Apart from coding, **technical + HR questions** are frequent:

### ⚙ Java / OOPs Concepts

- Difference between **Interface and Abstract Class**.
  - Explain **Polymorphism (Compile-time vs Runtime)** with example.
  - Difference between **Composition and Inheritance**.
  - What is **constructor chaining**?
  - Difference between **HashMap and ConcurrentHashMap**.
  - Explain **Thread Pool** and why it's used.
- 

### 📊 DBMS

- What is **primary key vs unique key**?
  - Explain **foreign key with cascade delete/update**.
  - Difference between **OLTP and OLAP**.
  - Explain **clustered vs non-clustered index**.
- 

### ⚡ Aptitude / Logical

Capgemini sometimes adds **Pseudo-code or Aptitude section**:

- Find output of given **pseudo-code snippet**.
  - Basic **math reasoning** (ages, percentages, probability, speed-distance).
- 

### 👤 HR Questions

- Introduce yourself.
  - Why Capgemini?
  - Tell us about your strengths/weaknesses.
  - Are you open to relocation/work in any technology?
  - Where do you see yourself in 5 years?
- 

## ◆ Tips

- ✓ Practice **LeetCode Easy/Medium**, Hackerrank Java/SQL sets.
  - ✓ Be clear with **Java OOPs + Collections + Threads**.
  - ✓ Revise **SQL joins, group by, aggregate functions**.
  - ✓ Be ready with **2 projects explanation** (tech stack, challenges, role).
- 

👉 Chetan, do you want me to prepare a **mock coding round (2 Capgemini-style coding questions)** for you so you can try solving them like in the real test?